

VertiCore Technical Architecture & Execution Model

Deep Technical Specification

1. Purpose of This Document

This document is a **deep technical companion** to the VertiCore Enterprise Architecture Specification. It is intended for:

- Platform engineers
- Infrastructure & SRE teams
- Security engineers
- Senior backend / ML engineers

The focus is on **how the system actually works at runtime**, including data flow, state transitions, execution semantics, and failure handling.

2. Runtime Execution Model (End-to-End)

2.1 Request Lifecycle

```
Client
  → GraphQL API
  → Control Plane
  → Temporal Workflow
  → Python Worker
  → Agent Graph
  → Tool Calls / LLM
  → Results
  → Audit + Persistence
```

Each arrow represents a **trust boundary crossing**.

3. Control Plane (Go) – Detailed Design

3.1 API Layer

- GraphQL schema defines allowed operations
- Strong typing enforces request shape
- AuthN via OAuth2 / mTLS

3.2 Session & Tenant Context

- Tenant ID injected into workflow metadata
- Context propagated via headers
- No tenant logic in agent code

3.3 Policy Enforcement (OPA)

Policy checks occur at: 1. Workflow start 2. Agent invocation 3. Tool execution 4. Memory promotion

OPA decisions are cached but revalidated on replay.

4. Temporal – Execution Semantics

4.1 Why Temporal

- Exactly-once execution
- Durable timers
- Deterministic replay

4.2 Workflow vs Activity

Component	Responsibility
Workflow	Orchestration, state
Activity	Side effects, compute

Agent execution always occurs inside **activities**.

5. Python Worker Architecture

5.1 Worker Process Model

- Stateless containers
- Horizontal scaling
- No durable storage

5.2 Execution Flow

```
Activity Start
  → Load Agent Definition
  → Apply Policies
  → Redact Input
  → Execute LangGraph
  → Validate Output
  → Emit Result
```

Workers can crash without data loss.

6. Agent Definition Model

Agents are defined declaratively:

```
agent:
  name: mortgage_analyzer
  graph: static
  nodes:
    - validate_input
    - analyze_financials
    - risk_score
    - route_decision
  tools:
    - credit_api
    - income_validator
```

Graphs are validated at build time.

7. LangGraph Internals

7.1 Static Graph Compilation

- Nodes compiled before deployment
- Routing limited to enumerated edges

7.2 State Management

- In-memory only
- Destroyed on activity completion

7.3 LLM Interaction

- Structured output enforced (JSON schema)
 - No free-form tool calls
-

8. Tool Execution Model

8.1 Tool Abstraction

- Tools are pure functions
- Side effects isolated

8.2 Execution Path

```
Agent Output
  → Tool Request
  → Policy Check
  → Tool Execution
  → Response Validation
```

Tools never receive raw prompts.

9. Memory Architecture (Technical)

9.1 Workflow Memory

- Stored in Temporal history
- Replay-safe

9.2 Vector Memory

- Qdrant collections per tenant
- RBAC-filtered queries

9.3 Audit Store

- PostgreSQL append-only tables
-

10. Human-in-the-Loop (Technical Semantics)

10.1 Trigger Conditions

- Confidence thresholds
- Missing data
- Policy violation

10.2 Suspension Mechanics

- Workflow yields
- Timer or signal waits

10.3 Resume Semantics

- New activity execution
 - Fresh agent context
-

11. Multi-Agent Coordination

11.1 Coordination via Temporal

- Fan-out/fan-in patterns
- Compensation logic

11.2 No Direct Agent Calls

- Prevents hidden coupling
-

12. Failure Modes & Recovery

Failure	Handling
Agent crash	Retry activity
Tool failure	Retry / fallback
Human delay	Durable wait

No partial execution states exist.

13. Observability & Debugging

- OpenTelemetry traces

- Deterministic replay
 - Per-node metrics
-

14. Security Threat Model (Technical)

Threats

- Prompt injection
- Tool misuse
- Data exfiltration

Mitigations

- Schema enforcement
 - Policy checks
 - No autonomous loops
-

15. Performance & Scaling

- Horizontal worker scaling
 - Bounded context windows
 - Token budgets
-

16. CI/CD & Validation

- Graph validation
 - Policy tests
 - Replay tests
-

17. Summary

VertiCore is designed as a **deterministic, policy-governed execution system** that embeds LLM reasoning without allowing it to control state, time, or authority.

This architecture enables safe AI agents in environments where failure is not an option.